

命令選択 — 複数の命令を 1 つに畳む

今日のゴール

「アドレスを計算してから読む」という 2 命令を、RV64 が持つ `offset(base)` 形式のロード 1 命令に畳み込む。 `fixed15` を通したまま、静的・動的の両方で命令数を減らす。

1 この回の位置づけ

- 最適化発展シリーズの第 3 回。前提は O1(測定基盤)
- `mycc.py` は書き換えない。 `optcc.py --passes isel` がパスとして差し込む
- 編集するのは `isel.py` だけ。 O2・O4・O5 とは独立している

2 いまのコンパイラは何を出しているか

変数 `s` を読むたびに、コンパイラは 2 命令を出している。

```
addi a0, s0, -24      # s のアドレスを計算して
lw   a0, 0(a0)        # そのアドレスから読む
```

コマ 4 以来の「`codegen_lval` でアドレスを作り、`ld/lw` で読む」という 2 段構えの設計から、まっすぐ出てきた形である。素直だが、無駄がある。

RV64 のロード命令は、もともと `offset(base)` という形を持っている。

```
lw   a0, -24(s0)      # 1 命令で同じこと
```

`addi` が要らない。変数を読むたびに 1 命令減るので、効きが大きい。

3 命令選択とは

いま行ったのは、「この計算にはこの命令が使える」という対応づけである。これを**命令選択** (instruction selection) という。

「s0 から -24 の位置にある 4 バイトを読む」という仕事
 → 素朴な割り当て: addi + lw (2 命令)
 → うまい割り当て: lw (1 命令)

同じ仕事に対して機械語の命令の選び方が複数あり、その中から良いものを選ぶ。CPU が持つアドレッシングモードや複合命令を活かすほど命令数が減る。

この回で扱う 3 つの置き換えは次のとおり。

前	後	効くところ
addi rD, rS, N	load rD, N(rS)	変数を読むたび
+		
load rD, 0(rD)		
addi rD, rS, N	store rV, N(rS)	変数へ書くとき
+		
store rV, 0(rD)		
li rC, 2 +	slli rD, rX, k	配列の添字計算
mul rD, rX, rC		

3 番目は、コマ 10 で作った「添字に要素サイズを掛ける」処理である。int なら 4 倍なので、掛け算ではなく 2 ビット左シフトで済む。掛け算は加算やシフトより数倍遅い命令なので、これも意味がある。

4 畳んでよいか — 消えるレジスタに注意

1 番目の置き換えは**必ず安全**である。畳む前も後も、rD には読み出した値が入る。

しかし 2 番目・3 番目は違う。

```
addi a1, s0, -16
sw    a0, 0(a1)      # → sw a0, -16(s0) に畳むと、a1 に何も入らなくなる
```

a1 を後で誰かが読むなら、畳んではいけない。実際、**畳んではいけない例**がすぐ見つかる。

```
addi sp, sp, -8      # スタックを確保して
sd   a0, 0(sp)       # そこへ積む
```

この形は `addi rD, rS, N + store rV, 0(rD)` にぴったり当てはまる。しかし畳んで `sd a0, -8(sp)` にすると、**sp** が動かなくなる。積んだつものの値は、次の `push` で上書きされて消える。

重要

パターンが一致することと、置き換えてよいことは別である。「その後でそのレジスタが使われるか」を確かめてから畳む。

そこで `is_dead_after(lines, pos, reg)` を作る。pos の後ろを 1 行ずつ見て、

見つけたもの	判定
reg を読む命令	まだ使われている → 畳めない
reg に書く命令	読まれる前に上書きされた → 畳んでよい
ラベル・分岐・call	ここで基本ブロックが終わる → 判断できない (安全側で畳まない)

3 行目が保守的なのがポイントである。ラベルの先は別の経路から来るかもしれないので、この単純な前向き走査では分らない。

補足

ここが **02・05** につながる。「ブロックを越えて、このレジスタが後で使われるか」を正しく知るには、フローグラフの上で**生存変数解析**を行う必要がある。それが 02(フローグラフ) と 05(生存解析) の主題である。

ただし、**正確にすれば畳める場所が増える、とは限らない**。実際に測ってみると、正確な生存解析を使ってもこの 3 つの畳み込みが増える箇所は **1 つも無い**。いまのコード生成が出す形では、たまたま「同じブロックの中で完結する」ものしか無いからである。解析を正確にすることと、最適化が効くことは別 —— これも **06** で扱う論点である。

5 実装

Step	実装対象	内容
1	<code>fold_load</code>	安全な畳み込み。まずこれだけで効果が出る
2	<code>is_dead_after</code>	「この後で使われないか」の判定
3	<code>fold_store / fold_mul_to_shift</code>	Step 2 を使って安全に畳む
4	<code>run</code>	変化がなくなるまで繰り返す (B1 と同じ)

`reads_reg / writes_reg`(命令がどのレジスタを読み書きするか) と正規表現はスケルトンで用意してある。

```
python3 ../optcc.py --passes isel ../01_measure/bench/matmul.c | grep -c slli
python3 ../optcc.py --passes ' ' ../01_measure/bench/matmul.c | grep -c "addi a0, s
```

6 テスト

```
python3 check.py      # 各置き換えの単体テスト (未実装は SKIP)
python3 golden.py     # fixed15 + ベンチマーク + 静的/動的の削減
```

`check.py` には「**push** の並びを畳まないこと」を確かめる項目がある。安全判定が働いていないと、ここで落ちる。

7 測ってみると

教員の実装では次のようになる。

ベンチマーク	静的：前	後	削減	動的：前	後	削減
bubble	311	281	9%	23209	20546	11%
fib_rec	78	74	5%	73994	69062	6%
loop_sum	70	65	7%	7840	7038	10%
matmul	386	351	9%	27321	24546	10%

strops	244	226	7%	77128	69086	10%
合計	1089	997	8%	209492	190278	9%

静的 8% 減に対し、動的は 9~11% 減。ループの中にある命令を消せているので、動的のほうがやや大きく効く。O1 で作った物差しが、ここで意味を持つ。

8 発展課題

1. **B1** と組み合わせる: `optcc.py --passes isel` と B1 のピープホールを 両方かけるとどうなるか。順番を変えると結果は変わるか
2. **addi** の畳み込み: `li rC, N + add rD, rX, rC` → `addi rD, rX, N` (N が 12 ビットに収まるとき) を追加する
3. 除算のシフト化: `li rC, 2 + div rD, rX, rC` は `srai` にできるか。負数のときに答えが変わるので注意 (S2 で見た「0 方向への切り捨て」)
4. 非隣接の畳み込み: 代入 `x = 式;` では、アドレス計算と `store` の間に式の計算が入るので隣り合っていない。これを畳むには何が必要か
5. どの置き換えがいちばん効いたか: 3 つを 1 つずつ無効にして測り、寄与を比べる

補足

コラム: 本物の命令選択は木の上で行う。教科書 (タイガーブック 9 章) の命令選択は、中間表現の木に対して「この形の部分木は、この 1 命令で実現できる」というタイルを敷き詰める (maximal munch や動的計画法)。今回は生成済みのアセンブリに対するパターン置換で同じ効果を出しているが、木の上で行えば「アドレス計算そのものを最初から作らない」ことができ、`is_dead_after` のような後追いの安全確認も要らなくなる。中間表現を導入する動機の 1 つが、ここにある。